第 3-2 讲 • 追一条数据流：记一笔账

> **模块 3 • 学会读代码** ｜ 建议用时 **40 分钟**（阅读 18 + 动手 22）
> 前置要求：完成 3-1（掌握读代码心法与项目地图）

本节目标

读完并做完这一讲，你能：

1. **独立画出**"点保存 → 数据进数据库"的完整链路（这次要画得更深，包含每一层在做什么）。
2. 看懂**校验层**：为什么要在主进程里再校验一遍（界面不是已经校验过了吗）。
3. 理解 **"失败时怎么办"**：错误是怎么从数据库一路传回界面的。
4. 解释一个真实的连锁反应：**保存一笔转账，为什么两个账户的余额都会变。**
5. 掌握追数据流的**终点判断**：什么时候算"追到底了"。

一、为什么要追"数据流"

1.1 现实比喻：快递追踪

你在网上买了个东西，物流信息是这样的：

```
已下单 → 商家发货 → 到达分拨中心 → 到达派送点 → 快递员派送 → 已签收
```

为什么你会看这个：因为东西没到的时候，你需要知道**卡在哪一环。**

追数据流是一模一样的：

```
界面点击 → preload 转发 → IPC 接住 → 校验 → 仓储层 → SQL → 数据库
```

当功能出问题时（"我保存了但列表里没有"），你能顺着这条链问：

- 界面真的发出请求了吗？（还是按钮没绑定）
- preload 转发了吗？

- IPC 接住了吗？（通道名字对不对）
- **校验通过了吗？**（很可能是被校验拦下了）
- SQL 执行了吗？

没有这条链路，你只能干瞪眼。有了它，你能像查快递一样定位问题。

1.2 追数据流的三条规矩

规矩	说明
从界面开始追	因为用户的问题总是从界面开始（"我点了没反应"）
每层只问三件事	① 谁调用它 ② 它做了什么 ③ 它把结果给了谁
追到 SQL 就到底	再往下是 SQLite 内部实现，**不属于你要理解的范围**

第三条很重要：初学者常想"SQLite 内部是怎么存字节的"——**那不是你的活。**就像你不必知道快递车用什么发动机。

二、完整的链路（含真实代码）

2.1 全景图



注意那个"跨进程边界"的标记：这是 2-1 讲说的"玻璃房"。**所有跨进程传递的数据都必须是"可序列化的"**（数字、字符串、数组、普通对象），**不能传函数、类实例、文件句柄。**

为什么值得知道：以后你传一个复杂对象过去报错时，就会想到这条限制。

2.2 第一站：界面侧（准备数据）

在 `RecordForm.tsx` 里，提交时构造了一个 `payload`（真实代码，节选）：

```
const payload = {
  date,                                                    // 日期字符串
  amount: cents,                                           // ★ 金额（分）
  type,                                                    // income / expense / transfer
  channel: type === 'transfer' ? 'transfer' : channel,     // 转账时渠道固定
  categoryId: type === 'transfer' ? null : childId || parentId || null,  // ★ 转账没有分类
  accountId: accountId || null,
  toAccountId: type === 'transfer' ? toAccountId || null : null,          // ★ 只有转账才有
  note: note.trim(),
  noteColor: null,
  tags
}
```

★ 标记的三处是关键的"业务规则"：

规则	代码体现
转账**没有分类**	`type === 'transfer' ? null : ...`
转账**必须有转入账户**	`type === 'transfer' ? toAccountId : null`
金额用**分**	`amount: cents`（前面已经转换过）

读代码的收获：**界面代码里藏着业务规则。**你在界面上看到的"转账时不显示分类选择框"，在代码里就是这三行三元表达式。

2.3 第二站：接口层（把关）

`main/ipc/records.ts`（真实代码）：

```
ipcMain.handle(IPC.recordsCreate, (_event, input: RecordInput): LedgerRecord => {
  assertReady()           // ① 数据库开着吗？
  validate(input)         // ② 参数合法吗？
  return repo.create(input)   // ③ 交给仓储层
})
```

只有三行。这就是"接口层只转发"的含义——它像个门卫：**查证件、登记，然后把活转给里面的人。**

2.4 为什么界面校验过了，主进程还要再校验一遍

这是一个初学者必问的问题：界面上明明已经限制了"金额必须是数字"，为什么主进程还要检查一次？

答案是：界面不可信。

三个理由：

理由	说明
界面是可以被绕过的	界面是"浏览器环境"，理论上可以被篡改（改内存、注入脚本）。**只有主进程的校验才是真的**
界面不可能覆盖所有情况	界面能拦"金额不是数字"，但拦不住"这个账户已经被删了"（那是数据库才知道的事）
校验是最后一道门	数据一旦进数据库，错误就会长期存在。**进库前必须把住**

这就是"纵深防御"（2-2 讲提过）：不指望一道防线，而是多层叠加。**界面的校验是为了用户体验（立刻提示），主进程的校验是为了数据正确性。**

2.5 校验层到底在检查什么（真实规则）

`validate()` 函数的规则清单（**这是我从代码里逐条抄出来的真实报错原文**，共 15 条）：

检查的代码	报错信息		
`!DATE_RE.test(input.date)`	日期格式不正确		
`!Number.isInteger(input.amount) \	\	input.amount < 0`	金额不正确
`input.amount === 0`	金额不能为 0		
`!['income','expense','transfer'].includes(input.type)`	记录类型不合法		
`!input.accountId`	请选择账户		
`!accountsRepo.get(input.accountId)`	账户不存在		
转账时 `!input.toAccountId`	请选择转入账户		
转账时 `toAccountId === accountId`	转出与转入账户不能相同		
转账时 `!accountsRepo.get(toAccountId)`	转入账户不存在		
非转账时 没选分类	请选择分类		
非转账时 `!category`	分类不存在		

检查的代码	报错信息
非转账时 `category.isArchived`	**该分类已归档，请重新选择**
非转账时 `category.type !== input.type`	分类类型与记录类型不一致
`input.note.length > 200`	备注请控制在 200 字以内
（更新时）`!repo.get(id)`	记录不存在

这张表就是"这个软件接受什么样的账"的完整定义。

注意「该分类已归档，请重新选择」这一条——它专门为一种场景准备：**用户在编辑弹窗里停留很久，期间在别的页面把那个分类归档了。**点保存时才被发现，提示他重新选。**这就是"读代码能读出产品设计"**：每条校验背后都有一个真实场景。

读代码的技巧：**校验函数是理解业务规则最快的地方。**想知道"能不能记负数"、"能不能用归档的分类"，看这里最快。

2.6 终点站：仓储层（真正写库）

`db/repositories/records.repo.ts`（真实代码）：

```
export function create(input: RecordInput): LedgerRecord {
  const id = randomUUID()      // 生成唯一 id
  const ts = Date.now()        // 生成时间戳
  getDb().prepare(
    `INSERT INTO records
     (id, date, amount, type, channel, category_id, account_id, to_account_id,
      note, note_color, tags, created_at, updated_at)
     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`
  ).run(id, input.date, input.amount, input.type, input.channel,
        input.type === 'transfer' ? null : input.categoryId,
        input.accountId,
        input.type === 'transfer' ? input.toAccountId : null,
        input.note, input.noteColor, JSON.stringify(input.tags ?? []), ts, ts)
  ...
}
```

四个值得注意的细节：

细节	为什么这么做
`randomUUID()` 生成 id	不依赖数据库自增，**导入/同步时不会冲突**
`?` **占位符**，不是拼接字符串	**防 SQL 注入**（如果直接把用户输入拼进 SQL，备注里写恶意内容就能删库）
`JSON.stringify(input.tags)`	标签是数组，存进 TEXT 字段要先转成字符串
只有转账才存 `to_account_id`	和界面侧规则一致——**这条规则在两层都体现，互相印证**

关于第 2 点，多讲一句：这是**安全编程的基本功**。看到任何把用户输入直接拼进 SQL 的代码，都是一个严重漏洞。**本项目全部用** `?` **占位符**，这是正确做法。

但有一个例外，非常值得学——我在核实代码时发现这两行（`categories.repo.ts`）：

```
const marks = ids.map(() => '?').join(',')
getDb().prepare(`SELECT id FROM categories WHERE parent_id IN (${marks})`).all(...ids)
```

这看起来像"把变量拼进 SQL"，但它其实是安全的标准做法。 关键区别：

	危险写法	这里的写法
拼进去的是什么	**用户的数据值**（如 `'张三'; DROP TABLE records`）	**只拼占位符** `?,?,?`
真正的值怎么进去	直接进 SQL 字符串	走 `.all(...ids)` **参数绑定**

为什么必须这么写：SQL 的 `IN (?, ?, ?)` 里，**问号的个数必须写死在语句里**，不能写成 `IN (?)` 再传一个数组。所以需要"有几个 id 就生成几个问号"——`ids.map(() => '?')` 正是干这个。

这是一条重要的读代码原则：

> **不要背"看到 `${}` 就是漏洞"，而要判断"拼进去的到底是数据还是占位符"。**

这就是"读懂代码"和"背规则"的区别。 前者能判断，后者会误报或漏报。

三、失败时会怎样（错误分支）

只看成功路径是不够的——真实软件里，一半的代码在处理"万一失败"。

3.1 错误的传播路线

```
仓储层抛错（比如账户不存在）
   ↓ 向上冒泡
接口层没有 try/catch → 继续冒泡
   ↓ 跨进程边界（Electron 自动把错误序列化）
preload 的 await 抛出异常
   ↓
界面侧的 try/catch 接住 → toast 弹出提示
```

对应到界面代码（真实写法）：

```
try {
  const saved = record
    ? await window.api.records.update(record.id, payload)
    : await window.api.records.create(payload)
  // 成功：继续处理
} catch (error) {
  toastError(error)     // ★ 把错误显示成用户能看懂的提示
}
```

`toastError` **的作用**：Electron 传过来的错误信息前面会带一长串技术前缀，`toastError` 会**把它剥掉，只留业务信息。**

它的真实实现只有三行（`stores\toast.store.ts`）：

```
export function errorMessage(error: unknown): string {
  const raw = error instanceof Error ? error.message : String(error)
  return raw.replace(/^Error invoking remote method '[^']+':\s*/, '').replace(/^Error:\s*/
}
export function toastError(error: unknown): void {
  useToastStore.getState().push(errorMessage(error), 'error')
}
```

逐行翻译：

代码	意思
`error instanceof Error ? error.message : String(error)`	拿到错误信息（不是 Error 对象就强转成字符串）
`.replace(/^Error invoking remote method '[^']+':\s*/, '')`	**用正则删掉 Electron 的技术前缀**
`.replace(/^Error:\s*/, '')`	再删掉可能残留的 `Error:`
`push(..., 'error')`	把干净的信息推给提示组件显示

实测数字：`toastError` 被用在 **16 个文件**里——**错误处理是普遍需求，不是某个功能的专属。**

这就是"好的错误处理"：用户不需要知道 IPC 是什么，他只需要知道**哪里不对、怎么改**。

3.2 为什么错误处理比你想象的重要

一个真实场景：用户在"记录"页打开编辑弹窗，停留了 10 分钟，中途在另一个窗口把那个分类归档了。然后他点保存——

- **有错误处理**：提示「该分类已归档，请重新选择」，用户改一下就好了。
- **没有错误处理**：程序卡住/白屏/静默失败，用户以为软件坏了。

这就是为什么校验函数里有一条 `if (category.isArchived) throw new Error('该分类已归档...')` —— 它专门为这种"状态在别处变了"的场景准备。

四、动手任务

步骤 1：亲手追一遍并画出链路

按顺序执行，**每步都要真的看到输出**：

```
cd "C:\Users\34123\Desktop\记账簿"

# ① 界面：构造 payload
Select-String -Path src\renderer\src\features\records\RecordForm.tsx -Pattern "const paylo

# ② 界面：发起请求
Select-String -Path src\renderer\src\features\records\RecordForm.tsx -Pattern "records.cre

# ③ 通道
Select-String -Path src\preload\index.ts -Pattern "recordsCreate"

# ④ 接口层
Select-String -Path src\main\ipc\records.ts -Pattern "recordsCreate" -Context 0,4

# ⑤ 校验
Select-String -Path src\main\ipc\records.ts -Pattern "function validate" -Context 0,6

# ⑥ 仓储层
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "export function cre
```

然后画出来（手画或打字都行）。画完对照 2.1 节的全景图，看有没有漏层。

步骤 2：读校验函数，列出业务规则

```
Get-Content src\main\ipc\records.ts | Select-String "throw new Error"
```

预期看到十几条报错。把它们翻译成人话，写成一张"这个软件接受的账"的规则表。

做完这个练习，你对这个软件的业务理解会超过 90% 的用户。

步骤 3：验证"转账"规则在两层都出现

```
# 界面层的转账规则
Select-String -Path src\renderer\src\features\records\RecordForm.tsx -Pattern "transfer' \

# 仓储层的转账规则
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "transfer' \?" | Sel
```

预期：两层都出现 `type === 'transfer' ? ... : null`。

领悟：**同一条业务规则，界面层负责"不给用户犯错的机会"，仓储层负责"万一还是来了，数据也不能脏"。**

步骤 4：找出所有"错误会怎么显示给用户"

```
Get-ChildItem src\renderer\src -Recurse -Include *.tsx | Select-String "toastError" | Grou
```

预期看到：界面各处都在用 `toastError`，**页面越多的地方用得越多。**

这告诉你：**错误处理是普遍需求**，不是"某个功能的事"。你以后写任何功能，都要想"失败了怎么告诉用户"。

步骤 5：追到底部的判断

```
Select-String -Path src\main\db\connection.ts -Pattern "export function" | Select-Object -
```

看一下 `connection.ts` 是干什么的（它是数据库连接的地方）。

然后问自己：要不要继续往下追（追进 `better-sqlite3` 这个第三方库的内部）？

答案：不要。 那是第三方库的实现，**不属于你要理解的范围。**你的地图边界是：`src\` **目录里的代码。**

"知道在哪停下"是读代码的重要能力——想搞懂每一层的人，永远读不完一个项目。

卡住了怎么办

现象	原因	怎么办
`Select-String` 输出乱码	控制台编码问题（1-2 讲提过）	加 `-Encoding UTF8`，或直接看 VS Code 里的文件
找不到 `payload`	变量名可能不同	先搜 `records.create`，看它上面几行
校验函数搜出来是空的	可能用了不同写法	搜 `throw new Error` 更保险
追着追着想追进 node_modules	边界感丢了	**记住：你的地图只到 `src\` 为止**

五、自测题

1. 完整说出"点保存 → 数据进数据库"经过哪些环节？

2. 界面上已经校验过金额了，为什么主进程还要校验一遍？

3. 仓储层的 SQL 里为什么用 `?` 占位符，而不是把值直接拼进字符串？

4. 保存失败时，错误是怎么从数据库一路传到用户眼前的？

5. 什么时候该停止往下追？

答案与解析

1. 界面（构造 payload）→ preload（转发）→ **IPC 接口层**（`assertReady` + `validate` + 转发）→ **仓储层**（生成 id、执行 INSERT）→ **SQLite**（写入磁盘）。 → 五站。其中"接口层"只把关不干活，"仓储层"才是真正写 SQL 的。

2. 因为**界面不可信**：① 界面属于浏览器环境，理论上可被篡改；② 界面**不可能知道所有情况**（比如"这个账户刚被删了""这个分类刚被归档了"——那要查数据库才知道）；③ **数据一旦进库，错误就会长期存在**，必须在入库前把住。 → 界面的校验是为了**用户体验**（立刻提示），主进程的校验是为了**数据正确性**。

3. 为了**防 SQL 注入**：如果直接把用户输入拼进 SQL，那么有人在备注里写一段恶意 SQL，就可能**删掉整个数据库。** → 用 `?` 占位符，数据库会把值当"纯数据"处理，不会当代码执行。**这是安全编程的基本功，**

看到拼接 SQL 的代码要警觉。

4. 仓储层抛错 → 接口层不捕获，继续向上 → 跨进程边界（Electron 自动序列化错误）→ preload 的 `await` 抛出 → **界面的** `try/catch` **接住** → `toastError` 剥掉技术前缀，显示成用户能看懂的话。 → 关键点：**用户看到的是业务信息（「该分类已归档」），不是技术细节。**

5. 当你到达 `src\` **的边界时**（比如进入 `node_modules` 里的第三方库）。 → 第三方库的内部实现**不属于你要理解的范围**——就像你不必知道快递车用什么发动机。**想搞懂每一层的人，永远读不完一个项目。**

六、术语表

术语	一句话解释
数据流 / Data Flow	数据从产生到落库经过的完整路径
payload	请求时携带的数据包
序列化 / Serialize	把对象转成可跨进程传输的形式
进程边界	两个进程之间的界限，跨界数据必须可序列化
校验 / Validate	检查输入是否符合规则，不合格就拒绝
占位符 / Placeholder	SQL 里的 `?`，用参数填充，防注入
SQL 注入	把用户输入当代码执行导致的攻击
错误冒泡 / Propagate	错误从底层一层层向上传递
`try/catch`	捕获并处理异常的语法结构
业务规则	这个软件"允不允许做某事"的规定
边界感（读代码）	知道读到哪一层就该停下的判断力

七、本讲与项目的对应

- 你追踪的链路：`RecordForm` → `preload` → `ipc\records.ts` → `db\repositories\records.repo.ts` → SQLite
- 你产出的东西：**一张完整的数据流图** + **一张业务规则表**（建议保留）
- **本讲不修改任何代码，不影响你的记账数据**
- 上一讲：3-1 读代码心法 ｜ 下一讲：**3-3 读第二遍：类型与契约**（为什么这个项目改一个字段要动好几个文件）

讲义版本：v1 ｜ 对应项目版本：0.1.0